Air Pollution Monitoring System

TITLE:

“IoT-Based Air Pollution Monitoring and Alert System Using ESP8266”

This fits well because IoT-based systems are commonly used for real-time air-quality sensing and alerts

Done By:

MUTHU KRISHNA - 24MEL033

PRIYANTH CS - 24MER044

SRI RAMAN G – 24MEL068

PRADEEP P – 24MELO41

Air Pollution Monitoring System

Project Overview

Air pollution is one of the major environmental problems affecting human health and the ecosystem. The **Air Pollution Monitoring System** is an IoT-based project developed to continuously monitor air-quality conditions.

The system uses sensors to detect changes in air quality and environmental parameters. An **ESP32 microcontroller** collects the sensor data, processes it and displays the information on an OLED/LCD screen. When the pollution level becomes high, the system activates an alarm.

The ESP32 can also transmit the collected data through Wi-Fi to an IoT platform, allowing users to monitor air quality remotely.

Main objectives:

- Monitor air pollution continuously.
- Detect harmful gases and smoke.
- Measure temperature and humidity.
- Provide an alert when pollution is high.
- Display sensor readings in real time.
- Enable remote monitoring using IoT.

Problem Statement

Air pollution is increasing because of vehicle emissions, industrial activities, burning of waste, construction activities and other sources.

Existing air-quality monitoring stations are often expensive and are generally installed only at selected locations. This makes continuous monitoring difficult in smaller areas.

Therefore, there is a need for a **low-cost, portable and IoT-enabled air pollution monitoring system** that can monitor environmental conditions and provide immediate warnings.

Problems with conventional monitoring:

- High installation cost.
- Large equipment size.
- Limited monitoring locations.

3. Proposed Solution

The proposed system uses an **ESP32** as the main controller.

An **MQ-135 sensor** is used to detect changes in air quality and gases, while a **DHT11/DHT22** sensor measures temperature and humidity.

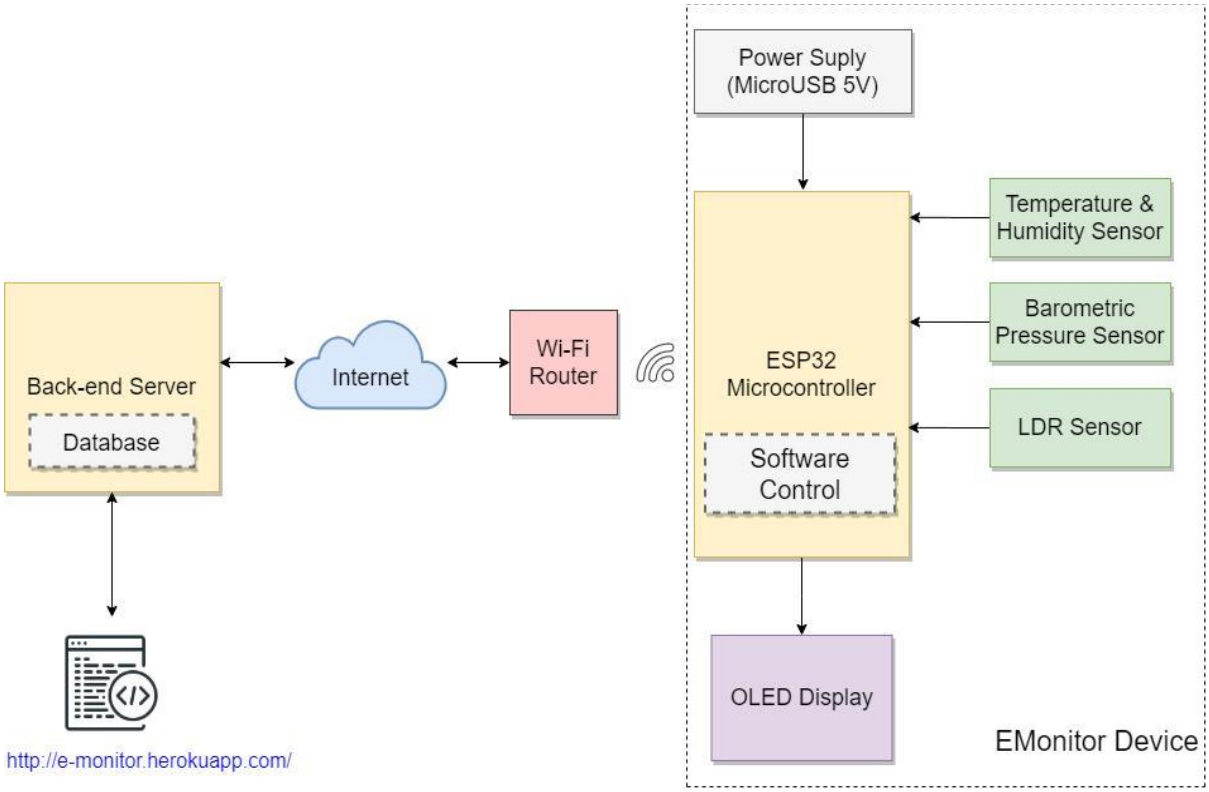
The ESP32 receives the sensor values and processes them. The results are displayed on an OLED/LCD. If the detected pollution value exceeds the selected threshold, a buzzer and warning LED are activated.

Using the ESP32's Wi-Fi capability, the data can also be uploaded to an IoT/cloud platform.

Advantages:

- Low cost.
- Compact size.
- Real-time monitoring.
- Easy to operate.
- Portable.
- IoT connectivity.
- Automatic warning system.

S.No	Component	Specification	Quantity	Function
1	ESP32	Wi-Fi enabled	1	Main controller
2	MQ-135	Air quality sensor	1	Detects air pollution
3	DHT11	Temperature & humidity	1	Environmental measurement
4	OLED	0.96-inch I²C	1	Displays readings
5	Buzzer	5V	1	Warning alarm
6	LED	Red/Green	2	Pollution status
7	Resistor	220 Ω	2	LED protection
8	Breadboard	Standard	1	Circuit assembly
9	Jumper wires	Male/Female	As required	Connections
10	USB cable	ESP32 compatible	1	Programming/power



5. Circuit Table

6. Circuit Design

Sensor connections

MQ-135:

- VCC → Power supply
- GND → GND
- AOUT → ESP32 analog input

DHT11:

- VCC → 3.3V
- GND → GND
- DATA → ESP32 GPIO

OLED:

- VCC → 3.3V
- GND → GND
- SDA → ESP32 SDA
- SCL → ESP32 SCL

Buzzer:

- Positive → ESP32 GPIO
- Negative → GND

Circuit operation

The MQ-135 senses changes in air quality. Its output is supplied to the ESP32. The ESP32 reads this value and determines the pollution condition.

The DHT11 provides temperature and humidity data. The OLED displays all important readings.

7. Algorithm

Step-by-step algorithm

1. Start.
2. Power ON the system.
3. Initialize ESP32.
4. Initialize MQ-135 sensor.
5. Initialize DHT11/DHT22 sensor.
6. Initialize OLED display.
7. Connect ESP32 to Wi-Fi.
8. Read the air-quality sensor value.
9. Read temperature.
10. Read humidity.

8. Coding

The ESP32 can be programmed using **Arduino IDE**.

The program performs the following functions:

- Reads MQ-135 sensor.
- Reads DHT11/DHT22.
- Displays values.
- Checks pollution threshold.
- Controls buzzer.
- Controls LEDs.
- Connects to Wi-Fi.
- Sends data to an IoT platform.

```
// ================================================
// AIR POLLUTION MONITORING SYSTEM
// ESP8266 NODEMCU + MQ-2 + 2 LEDs + BUZZER
// ================================================
```

```
// ================================================

// PIN DEFINITIONS

// ================================================


// MQ-2 Analog Output -> A0

#define MQ2_PIN A0


// Green LED -> D1 / GPIO5

#define GREEN_LED 5


// Red LED -> D2 / GPIO4

#define RED_LED 4


// Buzzer IO -> D6 / GPIO12

#define BUZZER_PIN 12


// ================================================

// GAS THRESHOLDS

// ================================================


// Adjust these values after checking

// your actual MQ-2 readings.


#define NORMAL_LIMIT 300

#define DANGER_LIMIT 600
```

```
// ================================================
// SETUP
// ================================================

void setup()
{
  Serial.begin(9600);

  // Configure pins
  pinMode(MQ2_PIN, INPUT);

  pinMode(GREEN_LED, OUTPUT);
  pinMode(RED_LED, OUTPUT);
  pinMode(BUZZER_PIN, OUTPUT);

  // Initial condition
  digitalWrite(GREEN_LED, LOW);
  digitalWrite(RED_LED, LOW);
  digitalWrite(BUZZER_PIN, LOW);

  Serial.println();
  Serial.println("====================================");
  Serial.println("   AIR POLLUTION MONITORING SYSTEM");
  Serial.println("====================================");

  Serial.println("MQ-2 Sensor Initializing...");
  Serial.println("Please wait...");
```

```
  // MQ-2 warm-up
  delay(10000);

  Serial.println("System Ready!");
  Serial.println("-----------------------------------");
}

// ================================================
// LOOP
// ================================================

void loop()
{
  // Read MQ-2 analog value
  int gasValue = analogRead(MQ2_PIN);

  // Display sensor value
  Serial.print("MQ-2 Gas Value: ");
  Serial.println(gasValue);

  // ================================================
  // NORMAL AIR
  // ================================================

  if (gasValue < NORMAL_LIMIT)
  {
```

```
  digitalWrite(GREEN_LED, HIGH);
  digitalWrite(RED_LED, LOW);
  digitalWrite(BUZZER_PIN, LOW);

  Serial.println("Air Quality : NORMAL");
  Serial.println("Green LED   : ON");
  Serial.println("Red LED     : OFF");
  Serial.println("Buzzer      : OFF");
}

// ================================================
// WARNING
// ================================================

else if (gasValue < DANGER_LIMIT)
{
  digitalWrite(GREEN_LED, LOW);
  digitalWrite(RED_LED, HIGH);
  digitalWrite(BUZZER_PIN, LOW);

  Serial.println("Air Quality : WARNING");
  Serial.println("Green LED   : OFF");
  Serial.println("Red LED     : ON");
  Serial.println("Buzzer      : OFF");
}

// ================================================
```

```
  // DANGER
  // ================================================


  else
  {
    digitalWrite(GREEN_LED, LOW);
    digitalWrite(RED_LED, HIGH);
    digitalWrite(BUZZER_PIN, HIGH);


    Serial.println("Air Quality : DANGER");
    Serial.println("HIGH GAS LEVEL DETECTED!");
    Serial.println("Green LED   : OFF");
    Serial.println("Red LED     : ON");
    Serial.println("Buzzer      : ON");
  }


  Serial.println("---------------------------------------");


  delay(1000);
}
```

9. System Working

When the system is switched ON, the ESP32 initializes all connected sensors.

The **MQ-135 sensor** continuously detects changes in air quality. The DHT11 measures temperature and humidity.

The ESP32 collects the sensor readings and processes them. The information is displayed on the OLED screen.

For example:

Temperature: 30° C
Humidity: 65%
Air Quality: Normal

When the pollution value increases beyond the selected threshold:

Air Quality: HIGH

The buzzer starts sounding and the warning LED turns ON.

If IoT connectivity is enabled, the readings are also transmitted through Wi-Fi to a cloud dashboard for remote monitoring.

---

10. Bill of Materials

S.No	Component	Qty.	Approx. Cost
1	ESP32	1	☐400
2	MQ-135	1	☐150
3	DHT11	1	☐100
4	OLED Display	1	☐150
5	Buzzer	1	☐20
6	LED	2	☐10
7	Resistors	2	☐5
8	Breadboard	1	☐100
9	Jumper Wires	1 set	☐80
10	USB Cable	1	☐100
	Total		**☐1,115**

11. Prototype Implementation

The prototype is constructed using a breadboard and jumper wires.

Implementation procedure

Step 1: Place the ESP32 on the breadboard.

Step 2: Connect the MQ-135 sensor to the ESP32.

Step 3: Connect the DHT11/DHT22 sensor.

Step 4: Connect the OLED display using I²C.

Step 5: Connect the buzzer and LEDs.

Step 6: Upload the program using Arduino IDE.

Step 7: Power the ESP32 using a USB cable.

Step 8: Observe the sensor readings.

Step 9: Test the system under normal and increased-pollution conditions.

Step 10: Verify the warning system.

12. Results & Performance Analysis

The developed prototype demonstrates the ability to monitor air-quality-related sensor readings continuously.

Expected results

Parameter	Result
Air-quality detection	Successfully monitored
Temperature measurement	Available
Humidity measurement	Available
Display	Real-time
Pollution alert	Automatic
IoT monitoring	Possible with Wi-Fi
Portability	Good
Cost	Low

The system provides:

- Continuous monitoring.

- Quick pollution-level indication.
- Automatic warning.
- Real-time display.
- Low-cost implementation.
- Possibility of remote monitoring.

Future Scope

The project can be improved by adding:

1. **PM2.5 and PM10 sensors**
2. Multiple gas sensors
3. GPS location tracking
4. Mobile application
5. Cloud data storage
6. Historical pollution graphs
7. Solar power supply
8. AI-based pollution prediction
9. Automatic ventilation control
10. Multiple monitoring stations

Conclusion

The **Air Pollution Monitoring System** is a low-cost IoT-based solution for real-time environmental monitoring. It can help identify increasing pollution levels and provide immediate alerts. With improved sensors and proper calibration, the system can be developed into a more accurate smart-city air-quality monitoring solution.